MAJOR PROJECT REPORT

on

Action Risk Calibrated Remediation: A Benchmark and Graph Model for Predicting the Blast Radius of Automated Remediation in Microservices

Submitted in partial fulfilment for the Award of Degree of

BACHELOR OF ENGINEERING
IN
INFORMATION TECHNOLOGY

BY

ABHISHEK KUMAR SINGH (160123737094)
LAKSH JAIN (160123737120)
YOGESH CHOUDHARY (160123737125)

Under the guidance of

Dr. RAMAKRISHNA KOLIKIPOGU
Professor

DEPARTMENT OF INFORMATION TECHNOLOGY
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)
(Affiliated to Osmania University; Accredited by NBA (AICTE) and NAAC (UGC),
ISO Certified 9001:2015), Kokapet (V), Gandipet (M), Hyderabad – 500 075
Website: `https://www.cbit.ac.in`

2026–2027

CERTIFICATE

This is to certify that the major project report entitled "Action Risk Calibrated Remediation: A Benchmark and Graph Model for Predicting the Blast Radius of Automated Remediation in Microservices" submitted to Chaitanya Bharathi Institute of Technology, in partial fulfilment of the requirements for the award of degree of B.E. (Information Technology) during the academic year 2026–27 is a record of original work carried out by Abhishek Kumar Singh (160123737094), Laksh Jain (160123737120) and Yogesh Choudhary (160123737125) during the period of study in the Department of Information Technology, CBIT, Hyderabad.

Project Guide
Dr. Ramakrishna Kolikipogu
Professor, IT Dept.
CBIT, Hyderabad.

Head of the Department
Dr. M. Venu Gopalachari
Professor, IT Dept.
CBIT, Hyderabad.

ABSTRACT

Automated incident-response tools for microservice systems restart, scale or drain a service to restore normal operation, and they decide whether to act by how confident they are about the root cause. Confidence says nothing about what the chosen action will cost: the action changes the traffic and load of the services around its target, and services that were healthy can start to fail or slow down. This report presents ARC (Action Risk Calibrated Remediation), a benchmark and a graph model that predict, before an action is applied, how much each service will be damaged by it. Damage is measured as the rise in the share of a service's requests that fail or become slow, over the 180 seconds after the action. We built a Kubernetes testbed with the Train Ticket application, injected faults, applied six remediation actions and recorded 1,795 controlled incidents, each labelled per service from exact request counters. A three-layer message-passing network over the service call graph, trained on the state of every service in the 60 seconds before the action, cuts the prediction error roughly in half against the best baseline in every test: 1.98 against 3.86 percentage points on new incidents of the same system, 2.86 against 3.91 on an unseen system, and 1.86 against 2.61 on an unseen fault. It names the most damaged service in 78% of incidents, scores all six candidate actions in about 25 ms, and drives a block, ask or allow gate that blocks 1.6% of safe actions and detects 84% of harmful ones. The model does not yet predict faults of a new kind, such as a four-times traffic surge, and these limits are reported. The dataset is prepared for open release.

Index Terms—AIOps, automated remediation, microservices, blast radius, graph neural networks, fault injection, benchmark dataset.

CONTENTS

Abstract **ii**

List of Figures **v**

List of Tables **vi**

1 Introduction **1**

1.1 Overview 1

1.2 Problem Definition 2

1.3 Aim of the Project 2

2 Literature Survey **3**

2.1 Paper 1 3

2.2 Paper 2 3

2.3 Paper 3 3

2.4 Paper 4 4

2.5 Paper 5 4

2.6 Paper 6 4

2.7 Paper 7 5

2.8 Summary of the Gap 5

3 System Requirement Specification **6**

3.1 Functional Requirements 6

3.2 Non-Functional Requirements 7

3.2.1 Speed and Memory Requirements 7

3.2.2 Usability Requirements 7

3.2.3 Reliability and Reproducibility Requirements 7

3.3 Software Requirements 7

3.4 Hardware Requirements 8

4 Methodology / Proposed System **9**

4.1 System Architecture / System Design 9

4.2 Testbed and Data Collection 10

4.2.1 Application and infrastructure . . . 10
4.2.2 Faults and actions . . . 10
4.2.3 Campaign design . . . 11
4.3 Damage Label . . . 11
4.3.1 Bad requests and measurement windows . . . 11
4.3.2 Damage measures . . . 12
4.3.3 Quality rules . . . 12
4.4 The ARC Dataset . . . 13
4.5 Damage Model . . . 13
4.5.1 Input features . . . 13
4.5.2 Graph network . . . 14
4.5.3 Training and final prediction . . . 14
4.6 Blast Radius, Action Choice and Decision Gate . . . 15
4.6.1 Blast radius . . . 15
4.6.2 Choosing an action . . . 15
4.6.3 Decision gate . . . 15
4.7 Experimental Setup and Evaluation Protocol . . . 15
4.7.1 Baselines . . . 15
4.7.2 Metrics . . . 16
4.7.3 Protocols . . . 16
4.8 Results . . . 16
4.8.1 Damage prediction . . . 16
4.8.2 Held-out faults and the noise floor . . . 17
4.8.3 Uncertainty, speed and decisions . . . 18
4.9 Validation and Known Limits of the Data . . . 19
4.9.1 Validity tests of the labels . . . 19
4.10 Second Application: Online Boutique . . . 22

5 Conclusion **24**
5.1 Conclusion . . . 24
5.2 Limitations . . . 24
5.3 Future Work . . . 25

References **26**

LIST OF FIGURES

4.1 Layered architecture of ARC. 9

4.2 Flow chart of the proposed system: (A) recording one episode and (B) using ARC at decision time. 10

4.3 Baseline window B and measurement window M around the action. . . . 12

4.4 Message passing: a service reads the state of its callers and of its callees. . 14

4.5 ARC against the best baseline in five tests. 17

4.6 ARC error on conditions never seen in training. 18

LIST OF TABLES

2.1 What related work provides and what ARC adds. 5

4.1 Faults and actions in the dataset (episodes). 11
4.2 Parts of dataset v1.0. 13
4.3 Prediction error (MAE, percentage points; lower is better). 17
4.4 ARC on conditions never seen in training. 18
4.5 Validity tests of the damage labels (Train Ticket, dataset v1.0). 20

Chapter 1

INTRODUCTION

1.1 Overview

Modern online services are built from many small programs called microservices. A user request, for example booking a train ticket, passes through a chain of services, and each one calls others. The map of which service calls which is the *call graph*. When one service fails or slows down, tools that watch the system, often called AIOps tools, try to repair it automatically. The common repairs are to restart the service, to start more copies of it, to give it more CPU, or to take it out of rotation (drain it). Recent systems use large language models or learned diagnosers to find the root cause of the incident and to choose the repair [12],[4].

These tools decide to act on how confident they are about the diagnosis. That confidence says nothing about what the repair will cost. Restarting one service stops the traffic it was serving; the services that call it see errors, and the services it calls suddenly receive fewer requests. A service that was healthy before the repair can therefore fail or slow down because of it. In an experiment on the Train Ticket application, restarting a single service after a network delay raised the share of failed or slow requests in several other services (Chapter 4).

This project addresses the missing step: predicting, before an action is applied, how much damage it will cause to each service. We call the system **ARC** (Action Risk Calibrated Remediation). ARC reads the recent state of every service, the call graph and the candidate action, and answers in milliseconds. This report describes the testbed we built, the dataset of 1,795 controlled incidents we recorded and labelled, the graph model that predicts the damage, the explanation and decision layers on top of it, and the evaluation.

This project can be used to:

- **Check an action before it runs:** an agent or an on-call engineer asks ARC what a restart, scale-up or drain would do to every service, and receives a block, ask or allow recommendation.
- **Choose the safest repair:** the same incident is scored for each of six candidate actions, and the one with the lowest predicted damage is suggested.
- **Explain the blast radius:** ARC shows which services will be hit, by how much, and through which call path.
- **Benchmark new methods:** the released dataset gives researchers per-service dam-

age labels and raw telemetry, which no existing public dataset records.

1.2 Problem Definition

Autonomous remediation tools do not predict the consequences of the actions they choose. The problem is to predict this damage before the action is applied. Given the state s of every service during the 60 seconds before the action, the call graph G, and a candidate action a with its target service, the model f must output for every service i the predicted rise $\hat{y}_i$ of the share of its requests that fail or become slow during the next 180 seconds:

$$f(G,s,a) \to \hat{y} = (\hat{y}_1, \ldots, \hat{y}_n),$$

where n is the number of services and $\hat{y}_i$ is measured in percentage points. The prediction must not use knowledge of the fault, must be fast enough for an agent to ask before every action, and must still hold on a system, a load and a fault type it was not trained on.

1.3 Aim of the Project

To develop and evaluate a learned model that predicts the per-service damage of automated remediation actions before they are applied, and to release a measured dataset that makes such models possible to build and compare. The objectives are:

1. to build a repeatable testbed that injects faults, applies remediation actions and records telemetry unattended;
2. to define a damage label that is measured from exact request counts and is robust to the choice of latency threshold;
3. to record a large, balanced dataset of incidents with all six actions applied to each fault condition;
4. to train a graph model and compare it with simple, tree-based and LLM-based baselines on new incidents, a new system, a new fault, a higher load and repeated incidents;
5. to add an explanation of the blast radius, a ranking of actions and a decision gate; and
6. to report the limits of the approach honestly, including the cases it does not predict.

Chapter 2

LITERATURE SURVEY

This chapter reviews the work closest to ours. Each entry gives the title, authors and venue, what the work does, and what it leaves open for this project. The works are grouped in our references at the end of the report.

2.1 Paper 1

Title: TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent with Multi-Modality Observation Data in Cloud-Native Systems
Authors: X. Zhang, Q. Wang, M. Li, Y. Yuan, M. Xiao, F. Zhuang, D. Yu
Publisher: IEEE Transactions on Services Computing, 2025 [12]
Description: The paper uses a tool-assisted large language model agent that reads metrics, logs and traces to localise the root cause of an incident and to classify the fault. It is the base paper of this project.
Gaps: It finds what failed but does not predict what a repair will do to the other services. Per-service damage of remediation is outside its scope.

2.2 Paper 2

Title: RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data
Authors: L. Pham, H. Zhang, H. Ha, F. Salim, X. Zhang
Publisher: WWW 2025; arXiv:2412.17015 [13]
Description: A benchmark of 735 failure cases from three microservice systems, with 15 baseline root-cause methods, covering metrics, logs and traces.
Gaps: It records faults and their root causes, not the consequences of repairs. It cannot be used to learn what an action does to the neighbouring services.

2.3 Paper 3

Title: MicroRemed: Benchmarking LLMs in Microservices Remediation
Authors: L. Zhang et al.
Publisher: arXiv:2511.01166, 2025 [4]

Description: A benchmark for language models that write executable remediation playbooks from a diagnosis, together with a multi-agent method (ThinkRemed).
Gaps: The focus is generating repairs. It does not predict the per-service impact of a repair before it is run.

2.4 Paper 4

Title: Safe Remediation as Risk-Constrained Intervention Decision in Microservice Systems
Authors: C. Dai, Z. Yan, C. Lei, Q. Li, L. Zhang
Publisher: arXiv:2607.20005, 2026 [1]
Description: Safe remediation is treated as a risk-constrained decision. Risk combines blast radius, reversibility and uncertainty, and a human-in-the-loop gate approves risky actions. The authors report 1,664 events and 8,320 decision records on Train Ticket, a 39% lower false-remediation rate and 17% fewer on-call escalations.
Gaps: Labels are per decision (resolved, partial or false remediation), not the damage to each service. We found no public code or data.

2.5 Paper 5

Title: Can AI Remediate Backend Failures Safely? GuardedAct with Blast-Radius-Aware Sandboxing
Authors: W. Cai et al.
Publisher: arXiv:2609.11264, 2026 [2]
Description: A language model proposes actions, a digital-twin sandbox estimates their blast radius, and a rollback-confidence gate decides. On DeathStarBench the authors report an 87.4% recovery rate and collateral damage falling from 25.6% to 5.2%, with about 8 s of added recovery time.
Gaps: Simulation adds delay to every decision. ARC targets a fast learned prediction (about 4 ms per action) rather than a sandbox run.

2.6 Paper 6

Title: Autonomous Incident Resolution at Hyperscale: An Agentic AI Architecture for Network Operations
Authors: A. Malik
Publisher: arXiv:2606.09122, 2026 [6]
Description: A multi-agent architecture with graduated autonomy, deployed at a major cloud provider, in which more than 90% of common incidents are resolved autonomously.
Gaps: No model of the consequences of individual actions on other components is reported.

2.7 Paper 7

Title: AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds
Authors: Y. Chen et al.
Publisher: arXiv:2501.06706, 2025 [9]
Description: A framework for evaluating AI agents on cloud operations tasks, with injected faults and an agent–cloud interface.
Gaps: It scores whether an agent solves the task. It does not measure, per service, how much collateral damage the agent's actions cause.

2.8 Summary of the Gap

Table 2.1 summarises what each line of work provides. None of them offers a learned, per-service prediction of the damage of a repair, trained on a released dataset of measured repair outcomes.

Table 2.1: What related work provides and what ARC adds.

Work	Finds root cause	Generates repair	Estimates repair risk	Per-service damage prediction
TAMO [12]	yes	–	–	–
RCAEval [13]	dataset only	–	–	–
MicroRemed [4]	–	yes	–	–
Dai et al. [1]	–	–	per decision	–
GuardedAct [2]	–	yes	sandbox (slow)	estimated in sandbox
ARC (this work)	–	–	learned, 4 ms per action	**yes, with released data**

Chapter 3

SYSTEM REQUIREMENT SPECIFICATION

3.1 Functional Requirements

1. The system must run a controlled incident unattended: check that the cluster is healthy, record a baseline, inject a fault, wait, apply one remediation action, measure, and store the result.
2. The system must support nine fault types (no fault, pod-kill, hung pod, network delay, packet loss, CPU squeeze, database delay, call cut and load surge) and six actions (no-op, restart-pod, rollout-restart, scale-up, cpu-bump and drain).
3. The system must plan campaigns in advance in blocks of seven episodes (one fault condition, one episode per action, no-op twice) so that action effects can be compared within the same kind of incident.
4. The system must record, for every episode, per-service request rates, errors, latency percentiles, CPU, memory and readiness every 5 seconds, the call graph with its failed calls, and every request the load generator sent.
5. The system must compute, for every service, the share of bad requests before and after the action, and the damage labels (mean, peak and recovery time), from exact request counters.
6. The system must calibrate the latency bar that defines a slow request on healthy episodes only, and check it on healthy episodes that were not used to fit it.
7. The system must flag episodes that were already saturated before the action, episodes with a machine stall, and services with too few requests to score.
8. The model must predict, for every service, the damage of a candidate action using only information available before the action.
9. The system must score all six candidate actions for one incident and recommend the one with the lowest predicted net damage.
10. The system must explain the blast radius: which services are hit, by how much, and through which call path from the target.
11. The system must return a block, ask or allow decision, using the predicted damage and the disagreement between model copies.

12. The system must clean up after every episode (faults, partitions, resized pods, scaled replicas) and stop safely after repeated failures.
13. The system must export every episode in an open format and provide a loader and checksums so that the dataset can be reused.

3.2 Non-Functional Requirements

3.2.1 Speed and Memory Requirements

A prediction must be fast enough for an agent to ask before every action. The graph model scores one action in about 4 ms and all six candidate actions in about 25 ms on a CPU. The complete tool, with five model copies, the tree blend and the explanation, takes about 350 ms and has not been optimised. Data collection is slower by design: one episode takes about 9 minutes and a campaign of several hundred episodes runs for days. The model itself is small (hidden vectors of 64 numbers) and needs no GPU for prediction.

3.2.2 Usability Requirements

An engineer or an agent must be able to ask ARC about an incident with one command and receive a short, readable answer: the decision, the predicted damage per service and the call chain behind it. A live viewer shows the state of the cluster and the call graph so that a campaign can be supervised. The released dataset has a datasheet, a schema description and a loader so that it can be used without knowing the testbed.

3.2.3 Reliability and Reproducibility Requirements

Campaigns must run unattended for days. The runner therefore checks the health of the cluster before every episode, repeats or discards an episode that fails its checks, stops after two failures in a row, and guards disk space. Plans are fixed before a campaign starts and every decision that changes the method is written in a decision log.

3.3 Software Requirements

- **Operating System:** Linux for the lab machine; Windows 11 with WSL2 for development.
- **Containers and cluster:** Docker, Kubernetes 1.34 (kind).
- **Applications under test:** Train Ticket (20 to 25 services) and Online Boutique (11 services plus a cache).
- **Fault injection:** Chaos Mesh 2.6.3, with in-place pod resizing for the CPU squeeze.

- **Telemetry:** OpenTelemetry Collector (span metrics and service graph), Prometheus (5 s resolution), cAdvisor, kube-state-metrics, Grafana.
- **Languages and libraries:** Python 3, NumPy, PyTorch, shell scripts.
- **Version control and release:** Git and GitHub for code; Zenodo and Hugging Face for the dataset.

3.4 Hardware Requirements

- **Data collection:** one workstation with an Intel Core i9-13900 processor (20 cores) and 32 GB of memory runs the whole cluster, the load generator and the telemetry stack; tens of gigabytes of free disk space for container images and metrics.
- **Modelling:** a laptop is enough; training and prediction run on the CPU.
- **Prediction at run time:** a single CPU core.

Chapter 4

METHODOLOGY / PROPOSED SYSTEM

4.1 System Architecture / System Design

ARC is organised in five layers (Figure 4.1). At the bottom, a Kubernetes testbed runs the application under test and an open-loop load generator that keeps the offered load constant even when the system slows down. Above it, the experiment harness injects faults, applies remediation actions and plans campaigns. The data pipeline turns the telemetry of each episode into features and damage labels. The ARC layer holds the graph damage model, the dynamic model, the blast-radius explainer and the decision gate. At the top, an interface lets an agent or an engineer ask about a candidate action before it is executed.

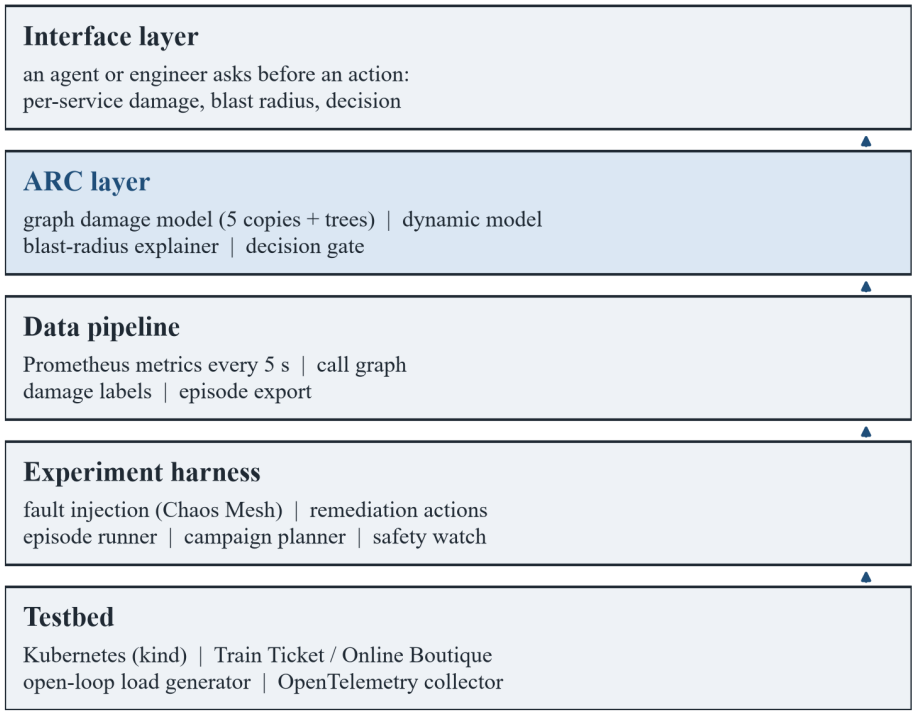


Figure 4.1: Layered architecture of ARC.

Figure 4.2 shows the two flows through this architecture. In the data-collection flow (A) one episode is recorded: the cluster is checked, a 120 s baseline is taken, a fault is injected, the system waits 60 to 300 s, the state of the last 60 s is snapshotted, one random action is applied, and the next 180 s are measured and labelled. Everything the model uses

is measured before the action; the label is measured after it. In the decision flow (B) the model reads the same pre-action state and the call graph and scores a candidate action; the damage per service becomes a blast-radius explanation and a decision.

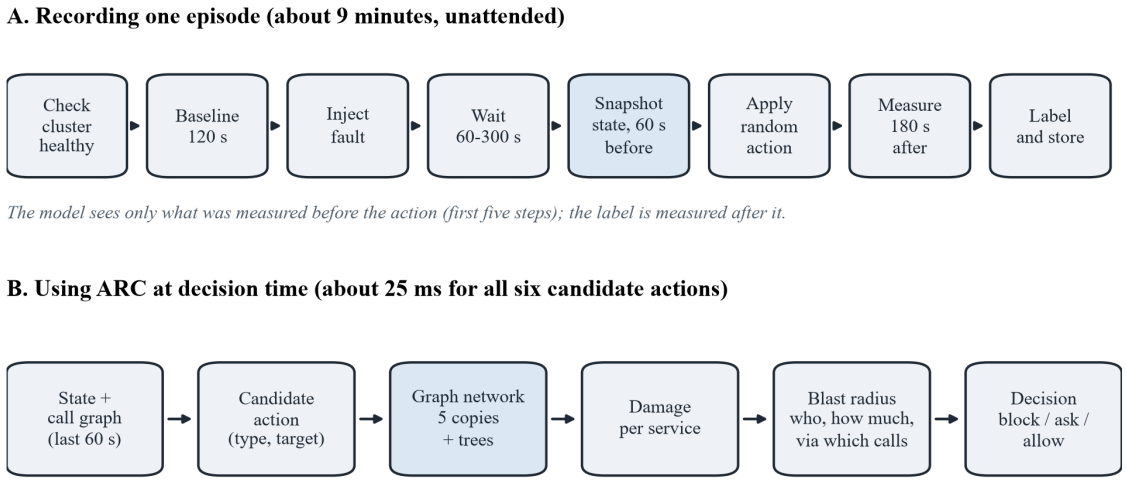


Figure 4.2: Flow chart of the proposed system: (A) recording one episode and (B) using ARC at decision time.

4.2 Testbed and Data Collection

4.2.1 Application and infrastructure

The main application is Train Ticket [20], a ticket-booking system of 20 services in its core form and 25 in its wide form, written mostly in Java. It runs on Kubernetes 1.34 inside kind on one workstation. OpenTelemetry spans reach Prometheus through the collector, which produces per-service request rates, latency histograms and the service call graph every 5 seconds; cAdvisor and kube-state-metrics supply CPU, memory and readiness. Chaos Mesh [22] injects the faults. A load generator sends nine kinds of user requests (search, orders, booking, payment, cancel and collection flows) at 4 or 8 requests per second with Poisson arrivals.

4.2.2 Faults and actions

Each episode injects one fault and applies one action 60 to 300 seconds later. Table 4.1 lists the nine fault types and six actions and how often each occurs in the dataset.

Table 4.1: Faults and actions in the dataset (episodes).

Fault	Episodes	Action	Episodes
none	273	no-op (control)	512
packet loss	467	restart-pod	259
network delay	434	rollout-restart	258
pod-kill	273	drain	258
hung pod	119	scale-up	255
CPU squeeze	61	cpu-bump	253
database delay	63		
call cut	63		
load surge	42		

The actions are: **restart-pod** (delete the pod so that a new one starts), **rollout-restart** (replace the pod through the deployment), **scale-up** (add a second copy), **cpu-bump** (raise the CPU limit in place), **drain** (take the instance out of rotation without stopping it) and **no-op** (do nothing; the control). Because the no-op is applied to every fault condition, the damage of an action can be separated from the damage of the fault itself.

4.2.3 Campaign design

A campaign is planned before it runs. It is cut into blocks of seven episodes that share one fault condition: one episode per action, and the no-op twice. Blocks never straddle the split between training and test data. The campaign runs unattended, 6 to 7 episodes per hour, with a health check before each episode, an automatic repeat of an episode that fails, a disk guard and a safety watch. The six parts of the dataset were recorded between 17 and 26 October 2026 on one Intel i9-13900 workstation with 32 GB of memory.

4.3 Damage Label

4.3.1 Bad requests and measurement windows

A request is *bad* if it fails or takes longer than a latency bar. We use three bars: a per-operation bar calibrated on healthy episodes (99th percentile of healthy latency, with a floor of 100 ms), and fixed bars of 500 ms and 1 s. The *bad-request share* of a service in a window is the number of its bad requests divided by all its requests.

For every action, two windows are used (Figure 4.3). The baseline window B is the 60 s before the action. The measurement window M starts 5 s after the action, lasts 180 s and is divided into six blocks $b_1, \ldots, b_6$ of 30 s.

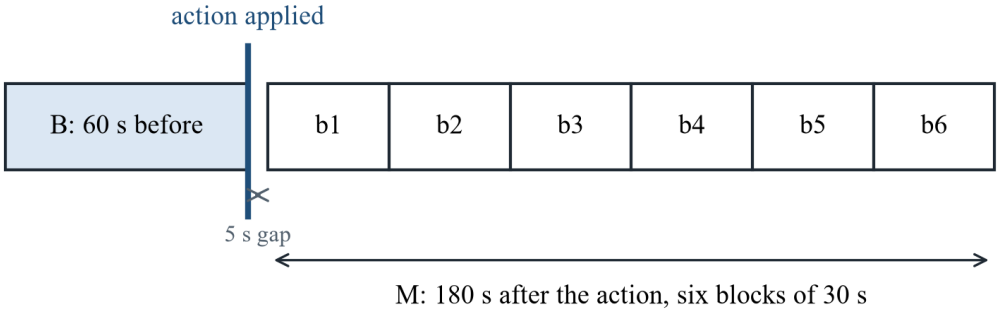


Figure 4.3: Baseline window B and measurement window M around the action.

4.3.2 Damage measures

The *mean damage* of service i is the change of its bad-request share between the two windows:

$$y_i^{\text{mean}} = p_i^M - p_i^B, \qquad p = \frac{\text{bad requests}}{\text{all requests}}. \tag{4.1}$$

For example, a service that went from 2% bad requests to 7% has a mean damage of 5 percentage points (pp). A block with few requests could show exactly 0% or 100%, so for the per-block shares half a request is added to the counts:

$$\tilde{p}_{ib} = \frac{\text{bad}_{ib} + 0.5}{n_{ib} + 1}. \tag{4.2}$$

The *peak damage* is the largest increase in any 30-second block, which exposes short, sharp harm that an average hides:

$$y_i^{\text{peak}} = \max_{b=1,\ldots,6} \tilde{p}_{ib} - \tilde{p}_i^B. \tag{4.3}$$

The *net effect* of an action is the sum of the mean damage over the services with enough traffic, that is at least 30 requests in both windows:

$$\text{net}(a) = \sum_{i \in E} y_i^{\text{mean}}, \qquad E = \{\, i : n_i^B \geq 30 \text{ and } n_i^M \geq 30 \,\}. \tag{4.4}$$

The net effect gives one score per action, which is what we use to compare actions. The labels also include the time a service takes to return to its baseline.

4.3.3 Quality rules

A service is scored only with 30 or more requests before and after the action. An episode in which at least 80% of user requests were already bad before the action is flagged as saturated and analysed separately. An episode in which more than a quarter of the measurement window was lost to a machine stall is flagged as stalled. Every incident is repeated across all six actions. The latency bar is calibrated only on healthy no-op episodes: for Train Ticket it was fitted on 12 episodes and checked on 8 that were not used for fitting.

4.4 The ARC Dataset

Dataset v1.0 contains 1,795 campaign episodes in one table with a column that names the part, plus 80 healthy calibration episodes that are used only to fit the latency bars (Table 4.2). Of the 37,790 labelled (episode, service) rows, 28,769 have enough requests to be scored. The dataset also records 4,696,050 individual user requests.

Table 4.2: Parts of dataset v1.0.

Part	Episodes	Services	Load	Faults
batch 1	701	20	4 req/s	none, pod-kill, delay, loss, CPU squeeze
wide1	500	25	4 req/s	none, pod-kill, hung pod, delay, loss
wide1-db	126	25	4 req/s	none, pod-kill, database delay
wide2	301	25	8 req/s	none, pod-kill, hung pod, delay, loss
repeat test	20	25	8 req/s	4 recipes repeated 5 times
held-out	147	25	8 req/s	none, pod-kill, call cut (3 links), load surge (3x, 4x)

For every episode the raw files are kept: per-service series every 5 s, the call graph with failed calls, exact request and latency-bucket counters, every request the load generator sent, Kubernetes events, the labels and the episode metadata. Any label can therefore be recomputed with another latency bar. The package has a datasheet, a schema description and a loader, checksums for every file, the licences (CC BY 4.0 for the data, MIT for the code), and host names and user paths were removed from the released files. It is prepared for release on Zenodo and Hugging Face.

4.5 Damage Model

4.5.1 Input features

Each service is described by 25 numbers: 13 state values measured in the 60 s before the action, a role code and the action type. The 13 state values are the request rate relative to the service's own healthy baseline, the error share, the 50th, 95th and 99th percentile latency relative to baseline, CPU use as a fraction of the limit, the share of CPU periods throttled, memory relative to baseline, whether the pod was ever not ready, the bad-request share before the action, an indicator that the service is the action target, and the shares of failed incoming and outgoing calls. All are relative to the service's own healthy level or bounded, so they mean the same on every machine, and the fault itself is never an input. The role code states the relation of the service to the action target: target, direct caller,

multi-hop caller, callee or unrelated. The action type is one of the six actions.

4.5.2 Graph network

A service's damage depends on its callers and callees, so the model works on the call graph. Each service starts from an embedding of its features x_i, role r_i and the action a:

$$h_i^{(0)} = \mathrm{ReLU}\big(W_0\,[x_i,\ r_i,\ a]\big), \tag{4.5}$$

a hidden vector of 64 numbers. Three message-passing layers follow (Figure 4.4). In each layer a service combines its own state with the mean state of its callers and the mean state of its callees, using separate weights for the two directions, and keeps its own state through a residual connection:

$$h_i^{(l+1)} = h_i^{(l)} + \mathrm{ReLU}\Big(W_s\,h_i^{(l)} + W_c\,\overline{h^{(l)}_{\mathrm{callers}(i)}} + W_e\,\overline{h^{(l)}_{\mathrm{callees}(i)}}\Big), \quad l = 0,1,2. \tag{4.6}$$

A small head maps the final vector to the predicted damage of service i.

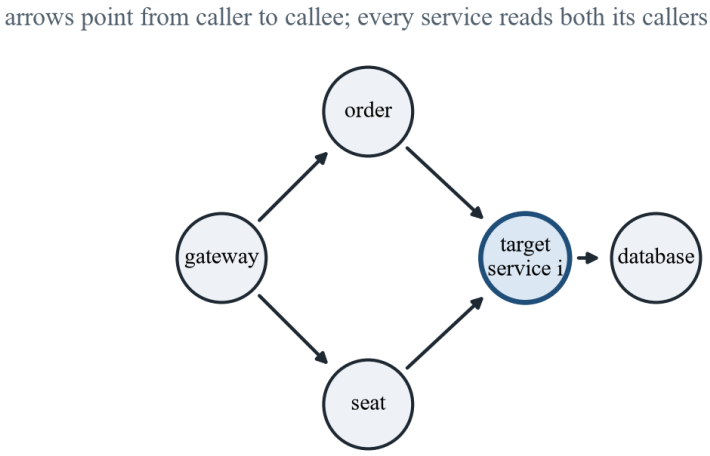


Figure 4.4: Message passing: a service reads the state of its callers and of its callees.

4.5.3 Training and final prediction

The network is trained with the L1 loss (mean absolute error) and the Adam optimiser with early stopping on a validation set made of whole blocks. Five random seeds give five model copies. The final prediction is a weighted ensemble of the graph model and a tree model:

$$\hat{y} = 0.9\,\hat{y}^{\mathrm{GNN}} + 0.1\,\hat{y}^{\mathrm{trees}}, \tag{4.7}$$

where $\hat{y}^{\mathrm{GNN}}$ is the mean of the five copies. The spread of the five copies is used as an uncertainty estimate.

4.6 Blast Radius, Action Choice and Decision Gate

4.6.1 Blast radius

Damage spreads through the call graph. Let $d_i(k)$ be the extra bad-request share of service i, k steps of 15 s after the action. A linear model fitted on the recorded trajectories describes how it moves:

$$d_i(k+1) = a\,d_i(k) + w_{dn}\,\overline{d_{\text{callees}(i)}(k)} + w_{up}\,\overline{d_{\text{callers}(i)}(k)} + g\,d_i(k_a) + u[a, \text{role}_i, k-k_a]. \quad (4.8)$$

The fit says that a service keeps most of its own damage ($a = 0.84$) and is pulled by the damage of the services it calls ($w_{dn} = 0.17$) about nine times more than by the services that call it ($w_{up} = 0.02$). The term g is the fault still present at the action, and u is the action's own effect for each role and step. The chain shown to the user is the shortest call path from the target to each service that is hit.

4.6.2 Choosing an action

The same incident is scored six times, once for each candidate action, and the safest action is the one with the lowest predicted net effect:

$$a^* = \arg\min_{a \in \mathscr{A}} \widehat{\text{net}}(a), \qquad |\mathscr{A}| = 6. \quad (4.9)$$

4.6.3 Decision gate

The gate turns the predicted net effect and the disagreement of the five copies, $\sigma(a)$, into a decision:

$$\text{decision}(a) = \begin{cases} \text{block} & \text{if } \widehat{\text{net}}(a) \geq 20, \\ \text{ask a human} & \text{if } \sigma(a) \geq \sigma_0, \\ \text{allow} & \text{otherwise.} \end{cases} \quad (4.10)$$

Twenty points of net damage is the blocking threshold. The disagreement threshold σ_0 is the 90th percentile of the copies' spread on training actions that would not be blocked, so it is set on training data only.

4.7 Experimental Setup and Evaluation Protocol

4.7.1 Baselines

ARC is compared with: **no damage** (predict zero), a **runbook average** (the average damage seen for the same action), a **role rule** (the average damage for the service's role relative to the target), a **dependency map** (damage assigned to services connected to the target in the call graph), **tree-based** models, a harmed-or-not **classifier**, and an **LLM judge** that

is asked to estimate the damage. All baselines were reimplemented and evaluated on the same data and split as ARC.

4.7.2 Metrics

The main metric is the mean absolute error (MAE) between the predicted and the measured damage, in percentage points, on scored services; lower is better. We also report how often the most damaged service is identified, the Spearman rank correlation between predicted and measured damage over all services (1 means identical order), the F1 score for harmed-or-not (a balance of missed harm and false alarms), the share of action pairs ranked correctly, and the block rate of the gate.

4.7.3 Protocols

Six protocols test different kinds of change.

- **In-system:** every fifth block of each plan is test data (5-fold); new incidents on the same system.
- **Unseen system:** train on batch 1 (20 services), test on wide1 (25 services, 5 of them new).
- **Unseen fault:** train on wide1, test on wide1-db (a slow database never seen in training).
- **Held-out faults:** train on everything except the held-out part, test on call cuts and load surges that were never seen.
- **Higher load:** train on the 4 req/s parts, test on wide2 (8 req/s).
- **Noise ceiling:** the same incident repeated five times, to measure the floor on any error.

4.8 Results

4.8.1 Damage prediction

Table 4.3 and Figure 4.5 give the error of ARC and of the best baseline in each protocol. ARC cuts the error roughly in half in every test.

Table 4.3: Prediction error (MAE, percentage points; lower is better).

Test	ARC	Best baseline	What changes
In-system (5-fold)	1.98	3.86 (role rule)	new incidents, same system
Unseen system	2.86	3.91 (trees)	20 to 25 services
Unseen fault	1.86	2.61 (trees)	slow database, never seen
Higher load, wide2 first half	2.78	4.11 (trees)	4 to 8 requests per second
Higher load, all wide2 (1 s bar)	2.12	4.17 (role rule)	4 to 8 requests per second

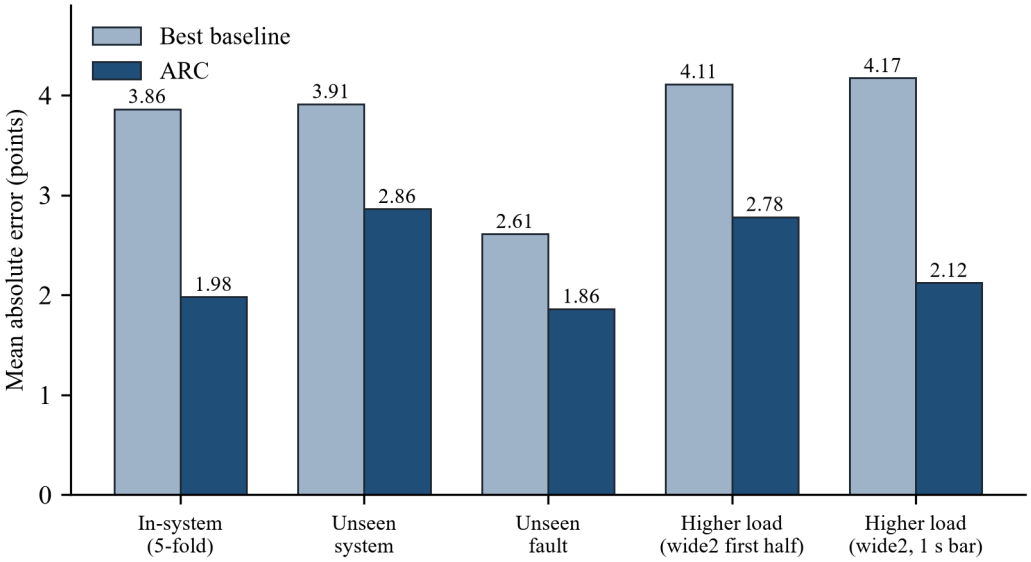


Figure 4.5: ARC against the best baseline in five tests.

ARC identifies the most damaged service in 78% of incidents, against 40% for trees. The trees, however, rank all services better (Spearman correlation 0.75 against 0.68), so ARC does not win on every metric. For harmed-or-not classification ARC reaches an F1 score of 0.77 against 0.75 for the classifier. Across latency bars from 50 ms to 1 s, ARC's error is 0.21 to 0.56 times the error of predicting no damage. On the batch-1 test split alone, ARC's error is 1.83 against 6.46 for the dependency map and 9.03 for the runbook average.

4.8.2 Held-out faults and the noise floor

Table 4.4 and Figure 4.6 show ARC on 147 episodes of fault types that were not in the training data. ARC is accurate on call cuts and no-fault episodes, but the average is dominated by the four-times traffic surge (error 22.16), a fault of a new kind with no failing component, which the model does not predict. Over all held-out episodes the error is 7.57, level with the best baseline (role rule, 7.51) and below predicting no damage (9.21).

Table 4.4: ARC on conditions never seen in training.

Condition	Episodes	ARC error	Worst service hit (share bad)
call cut ts-assurance-service	21	0.98	36%
call cut ts-config-service	21	4.65	67%
call cut ts-order-other-service	21	3.24	40%
traffic surge 3x	21	6.09	33%
traffic surge 4x	21	22.16	40%
no fault	21	3.20	44%
pod-kill	21	5.51	41%
all held-out	**147**	**7.57**	**42%**

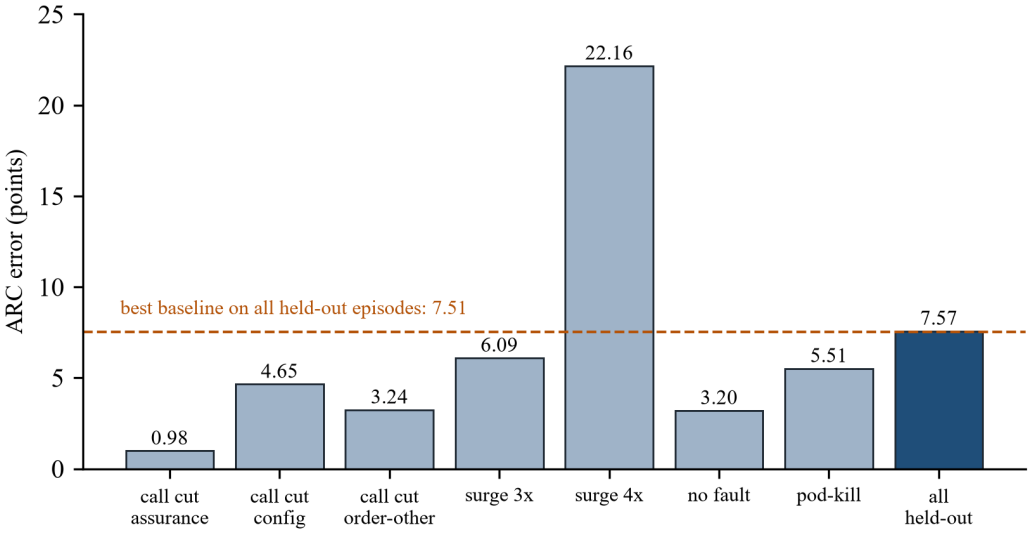


Figure 4.6: ARC error on conditions never seen in training.

The same incident repeated five times does not give the same numbers: total damage varies by 12 to 27%. No predictor can therefore score below 3.08 points on these incidents; ARC is 2.1 points above this floor and the trees are 7.2 points above it.

4.8.3 Uncertainty, speed and decisions

Uncertainty. When the five model copies disagree the prediction is more often wrong. In the 20% of episodes with the highest disagreement the error is 3.7 against 1.3 (in-system), 7.2 against 2.0 (unseen system) and 4.5 against 1.2 (unseen fault).

Speed. The model scores one action in 4 ms and all six in about 25 ms; the full tool takes about 350 ms before optimisation. The LLM judge needs 3.6 s and is no better than predicting no damage.

Explanation, ranking and gate. The blast-radius explanation reaches a hit-set F1 of 0.74, against 0.64 for a dependency map. Ranking the six actions by predicted net damage puts

86% of action pairs in the right order, against 80% for a runbook-style average. The gate blocks 1.6% of safe actions and detects 84% of harmful ones.

4.9 Validation and Known Limits of the Data

What has been checked. (i) The latency bar was fitted on healthy episodes only and checked on healthy episodes that were not used to fit it (Train Ticket: fitted on 12, checked on 8). (ii) Labels are reported at three bars, so the results do not depend on one threshold. (iii) Repeating the same incident gives the measurement noise (12 to 27%, floor 3.08). (iv) Plans, discard rules and scoring rules were fixed before the runs and are applied by code. (v) Every released file has a checksum, the archives pass a gzip test, the loader returns the expected counts for every split, and a scan found no host names or user paths in the released files.

4.9.1 Validity tests of the labels

No public benchmark records the damage of a repair, so the labels cannot be checked against someone else's answers. We therefore tested them four ways on the released tables (campaign episodes only; values in percentage points). Table 4.5 summarises the tests and each is explained below.

Table 4.5: Validity tests of the damage labels (Train Ticket, dataset v1.0).

Test	Result	Verdict
1. Negative control	No fault and no-op (76 episodes, 1,288 scored rows): 0.2% harm verdicts, mean damage 0.005 pp. Healthy calibration episodes: 0 of 1,406 rows.	Labels do not invent damage
2. Dose–response	Within each target service, a higher level of packet loss or delay gives a higher bad-request share before the action (7 of 7 targets each; median Spearman 0.86 and 0.58).	Labels follow a known cause; weak for database delay
3. Two instruments	Gateway counters against the load generator's request log (1,736 episodes): Pearson 0.95 and Spearman 0.86 overall, Spearman 0.95 where a real effect exists. The counters read lower when damage is large.	Agree; service-side labels are conservative
4. Mechanism	After a drain, direct callers of the target are 66 pp worse than in the no-op of the same block, callees and unrelated services 0 pp.	Damage follows the call graph

Test 1: negative control. In healthy conditions an action-free episode should show no damage. In the 76 campaign episodes with no fault and a no-op, 3 of 1,288 scored service rows (0.2%) carry a harm verdict, and the 95th percentile of the absolute change is 0.4 pp. The episode means are indistinguishable from zero (t-test, $p = 0.89$). The 80 healthy calibration episodes give 0 harm verdicts in 1,406 rows. For contrast, in no-fault episodes with a drain, 19% of rows are harmed and the mean damage is 14.5 pp. The labels therefore stay at zero when nothing happens and react when something does.

Test 2: dose–response. A stronger fault should hurt users more before any action is taken, so we compared the fault level with the bad-request share users saw in the 60 s before the action (which does not depend on the action). Fault levels were dosed separately for each target service, so the test is run within each target. For packet loss, all 7 target services show a positive relation (Spearman ρ from 0.56 to 0.91, median 0.86); for network delay all 7 are positive (median 0.58), although one target (ts-travel2-service, 26 episodes) is not significant ($\rho = 0.15$, $p = 0.47$). A CPU squeeze that leaves the service 1 times its recent use gives 72% bad user requests against 13% at 4 times its use, and a 4-times load surge gives 41% against 6% at 3 times. Database delay is the exception: the relation is weak ($\rho = 0.21$ over all targets, $p = 0.10$), so database-delay levels should not be used as a severity scale. Episodes with no fault show 0.4% bad requests. Fault levels remain

machine-specific (see the known issues below).

Test 3: two independent instruments. The label comes from service-side counters, while the load generator keeps its own log of every request it sent. If the labels are real, the two must tell the same story. We compared the 1 s-bar damage computed from the gateway service's counters with the same quantity computed from the client log, over 1,736 episodes. The agreement is high (Pearson 0.95, Spearman 0.86; 81% of episodes within 5 pp and 87% within 10 pp). Where a real effect exists (either instrument above 5 pp, 653 episodes) the rank agreement is higher (Spearman 0.95). In the non-saturated episodes of this group the sign agrees in 96% of cases, and both instruments rank the actions in the same order: drain (48.0 against 52.0 pp), restart-pod (26.6 against 27.5), rollout-restart (14.4 against 26.3), then scale-up, cpu-bump and no-op near zero. The service-side counters, however, read lower than the clients see when damage is large (mean difference -4.6 pp over the 653 episodes, and about 13% of all episodes differ by more than 10 pp, most often those with a pod-kill (33% of them) or a CPU squeeze (26%), possibly because requests can be lost before they are counted). The service-side labels are therefore a conservative, lower-bound estimate of what users experience, most clearly for rollout-restart.

Test 4: mechanism. Damage should appear in the services connected to the target, not elsewhere. For each action we built the call graph of the episode, labelled every service by its position relative to the target, and subtracted the no-op of the same block (non-saturated episodes only). After a drain, the direct callers of the target are 66.4 pp worse than in the no-op (96% harm verdicts) and the callers two or more hops away 68.3 pp worse (96%), while the target's callees (+0.2 pp) and unrelated services (-0.3 pp) do not change. A pod restart gives the same pattern with smaller effects (direct callers +27.5 pp, farther callers +48.8 pp, callees and unrelated services below 1 pp), and a rollout restart is weaker still (+9.6 and +24.1 pp). Scale-up and cpu-bump leave every group within 2 pp of zero. Damage thus flows to the services that call the target, which is the direction found by the fitted spread model (the pull from the services a service calls is 0.17, from its callers 0.02), and it does not reach services outside the call chain.

Known issues. The datasheet lists them; the ones that change results are:

1. **Saturation.** In 389 of 1,795 episodes users were already at least 80% bad before the action. They are flagged and should be excluded, or the 1 s bar used, when ranking actions.

2. **Drift in wide2.** Two tables read by the cancel flow were not cleaned, so the cancel flow slowed from 77 to 131 ms during wide2. Labels after block 24 are inflated for that flow; the first half of wide2 or the 1 s bar should be used.

3. **One system, one machine, one copy per service.** Fault levels were dosed on this machine and are not a measure of severity.

4. **Short bursts are invisible.** The label averages over 180 s; a burst of a few seconds stays in the raw requests only.

What has not been done yet. The labels measure what the users see (the share of bad

requests); no operator has judged whether a given rise matters. All data of version 1.0 was recorded on one machine with one pipeline, so independent replication is still open (the Online Boutique campaign is a first step). Confidence intervals for the results of Table 4.3 (a bootstrap over blocks) and the ablations of the model have not been computed yet.

4.10 Second Application: Online Boutique

To test whether the pipeline works beyond one application, the testbed was ported to Online Boutique, a Google demonstration shop of 11 services and a cache. The port needed several changes. Three services (ad, cart and shipping) are written in Go, C# and Java and emit no server-side spans, so they are named through the service graph from their callers' client spans and their labels are computed from the callers' view of the calls (caller view, threshold 100 ms). Draining a service is done with a Chaos Mesh network partition because Online Boutique has no service registry. A restart whose effect lasts about 10 s is invisible in a 180 s label. The latency bar was calibrated at 8 requests per second on healthy episodes (0 false harms in 42 held-out service verdicts, and every service receives a verdict).

A campaign of 280 episodes (40 blocks of 7, eight fault conditions, six actions) was planned in advance. All 280 planned slots were recorded in 283 episode folders: three episodes were discarded (two failed drains and one fault that could not be verified) and the runner repeated each of them automatically. Two harness faults were found by the safety rules and fixed during the run: a failed drain left its network partition behind, and a CPU squeeze that was followed by a scale-up left a pod throttled; five episodes affected by the second fault are kept but excluded, and the five slots were re-run with the fix (all clean), which gives 280 clean episodes. Saturation in this campaign (59 of the 280 clean episodes, 21%) comes almost only from network delay (37) and hung pods (21), which is a finding about those faults; it occurred once with packet loss and never with CPU squeeze, pod-kill or no fault. The four validity tests of Section 4.9 were repeated on the final data (280 clean episodes, 2,726 scored service rows). **Negative control:** 10 non-saturated no-fault no-op episodes (100 rows) show 0 harm verdicts and a mean change of -0.003 pp, and the 40 healthy calibration episodes (279 rows) show none. **Dose–response:** a delay level of 25 gives 24.9% bad user requests before the action against 12.5% at level 10 (Mann–Whitney $p = 0.018$), and 15% packet loss gives 8.6% against 0.4% at 5% ($p < 10^{-10}$); episodes with no fault show 0.07%. **Two instruments:** the frontend counters and the load generator's log agree (280 episodes: Pearson 0.98, Spearman 0.89; the 53 episodes with a real effect: Spearman 0.99, sign agreement 98%), but as in Train Ticket the counters read lower when damage is large (drain: 28.4 against 44.4 pp). **Mechanism:** after a drain, direct callers of the target are 45.2 pp worse than in the no-op of the same block (86% harm verdicts) and farther callers 10.7 pp worse, callees do not change (0.0 pp), and unrelated services change little (+2.3 pp, 9% harm verdicts, which may include services that depend on the target through calls our edge list does not show). Restarts move the direct callers by under 1 pp, consistent with the limit stated above that a burst of about 10 s is invisible in a 180 s label, and scale-up and cpu-bump leave callers and unrelated services within 1.5 pp of zero (scale-

up lowers the target's own damage by 5.8 pp). The Online Boutique results are therefore treated as a replication with stated limits, and Train Ticket remains the main benchmark.

Chapter 5

CONCLUSION

5.1 Conclusion

Automated remediation tools decide whether to act from their confidence in a diagnosis, and nothing tells them what the action will cost the services around its target. ARC fills this gap. We built a repeatable testbed, recorded 1,795 controlled incidents on the Train Ticket application with all six remediation actions applied to each fault condition, and labelled each service by the rise in its share of failed or slow requests, computed from exact request counters. A three-layer graph network over the call graph, combined with a tree model, predicts this damage before the action is applied.

The model cuts the prediction error roughly in half against the best baseline in every test: 1.98 against 3.86 percentage points on new incidents, 2.86 against 3.91 on an unseen system, 1.86 against 2.61 on an unseen fault, and 2.78 against 4.11 at twice the load. It names the most damaged service in 78% of incidents, ranks 86% of action pairs correctly, explains the blast radius with a hit-set F1 of 0.74, and drives a gate that blocks 1.6% of safe actions and detects 84% of harmful ones. Scoring all six actions takes about 25 ms, so an agent can ask before every action. The disagreement of the five model copies is a useful warning: the episodes with the highest disagreement have several times the error.

The dataset, with raw telemetry, labels, splits, a datasheet and a loader, is prepared for open release, and a second application, Online Boutique, has been used to test whether the pipeline carries over.

5.2 Limitations

- The evaluation covers one main application on one machine, with one copy of each service. A second application is under evaluation and is a replication with stated limits.
- Generalisation to new kinds of incident is still weak: over all held-out episodes the error (7.57) is level with the best baseline (7.51), and it reaches 22.16 under a four-times traffic surge, which has no failing component.
- The trees rank all services better than ARC (Spearman 0.75 against 0.68), and some action rankings change with the latency bar.
- Labels average over 180 s, so bursts of a few seconds are not seen. They measure

what users see, not whether an operator would consider the rise important, and the service-side counters read lower than the load generator's own log when damage is large (conservative labels, most clearly for rollout-restart).

- Saturated episodes (389 of 1,795) and the drift in the second half of wide2 need care when the data is reused.

5.3 Future Work

- **Model:** use temporal telemetry, the age of the fault, the number of replicas and traffic changes as inputs; evaluate calibrated uncertainty intervals (conformal prediction [25],[26]); predict the distribution of latency rather than a single share.
- **Integration:** connect ARC to a root-cause diagnosis agent and to a remediation executor; optimise the complete tool toward a latency below 50 ms; monitor baseline drift.
- **Data and evaluation:** finish the Online Boutique campaign and release it as version 1.1; repeat the four validity tests on the Online Boutique data, add confidence intervals and ablations to the results; use ARC as a guard on public agent benchmarks, for example the Online Boutique incidents of [7] or SREGym [10], and report how much collateral damage the gate removes and how much recovery success it keeps; keep the labels of the held-out part behind a scoring script so that new methods can be compared on an untouched test set; release the dataset on Zenodo and Hugging Face. No public benchmark scores the per-service damage of a repair, which is why these tests are proposed rather than reused.

REFERENCES

[1] C. Dai, Z. Yan, C. Lei, Q. Li, L. Zhang, "Safe Remediation as Risk-Constrained Intervention Decision in Microservice Systems," arXiv:2607.20005, 2026.

[2] W. Cai et al., "Can AI Remediate Backend Failures Safely? GuardedAct with Blast-Radius-Aware Sandboxing," arXiv:2609.11264, 2026.

[3] L. Bindschaedler, "Rebooting Microreboot: Architectural Support for Safe, Parallel Recovery in Microservice Systems," Proc. ARCS, 2026. arXiv:2604.09963.

[4] L. Zhang et al., "MicroRemed: Benchmarking LLMs in Microservices Remediation," arXiv:2511.01166, 2025.

[5] L. Zhang et al., "E2E-REME: Towards End-to-End Microservices Auto-Remediation via Experience-Simulation Reinforcement Fine-Tuning," FSE 2026. arXiv:2604.11094.

[6] A. Malik, "Autonomous Incident Resolution at Hyperscale: An Agentic AI Architecture for Network Operations," arXiv:2606.09122, 2026.

[7] J. Qi et al., "Can LLMs Really Recover Microservice Failures? A Recovery-Aware Evaluation of Diagnosis-to-Action Reasoning," arXiv:2607.04623, 2026.

[8] S. Fatima et al., "CRRL: A Causality-Based Reinforcement Learning Framework for Autonomous System Recovery," arXiv:2607.03177, 2026.

[9] Y. Chen et al., "AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds," arXiv:2501.06706, 2025.

[10] J. Clark et al., "SREGym: A Live Benchmark for AI SRE Agents with High-Fidelity Failure Scenarios," arXiv:2605.07161, 2026.

[11] Y. Chen et al., "STRATUS: A Multi-agent System for Autonomous Reliability Engineering of Modern Clouds," arXiv:2506.02009, 2025.

[12] X. Zhang, Q. Wang, M. Li, Y. Yuan, M. Xiao, F. Zhuang, D. Yu, "TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent with Multi-Modality Observation Data in Cloud-Native Systems," IEEE Trans. Services Computing, 2025. arXiv:2504.20462.

[13] L. Pham, H. Zhang, H. Ha, F. Salim, X. Zhang, "RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data," arXiv:2412.17015, 2024.

[14] A. Fang et al., "Rethinking the Evaluation of Microservice RCA with a Fault Propagation-Aware Benchmark," FSE 2026. arXiv:2510.04711.

[15] F. Gao et al., "Are GNNs Actually Effective for Multimodal Fault Diagnosis in Microservice Systems?," arXiv:2501.02766, 2025.

[16] I. Buljić, "Does Graph Structure Earn Its Place in Microservice Root-Cause Analysis? A Controlled Study on RCAEval," arXiv:2609.27069, 2026.

[17] Y. Gan et al., "Sage: Leveraging ML to Diagnose Unpredictable Performance in Cloud Microservices," arXiv:2112.06263, 2021.

[18] S. Zhang et al., "Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis," arXiv:2407.01710, 2024.

[19] L. Nagalapatti et al., "Robust Root Cause Diagnosis using In-Distribution Interventions," ICLR 2025. arXiv:2505.00930.

[20] X. Zhou et al., "Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study," IEEE Trans. Software Eng., vol. 47, no. 2, pp. 243–260, 2021 (Train Ticket).

[21] A. Basiri et al., "Automating Chaos Experiments in Production," ICSE-SEIP, 2019. arXiv:1905.04648.

[22] Chaos Mesh, a cloud-native chaos engineering platform for Kubernetes, `https://chaos-mesh.org`.

[23] S. Smith et al., "Benchmarks for End-to-End Microservices Testing," IEEE SOSE, 2023. arXiv:2306.05895.

[24] Y. Feng et al., "CORA: Conformal Risk-Controlled Agents for Safeguarded Mobile GUI Automation," arXiv:2604.09155, 2026.

[25] J. Lekeufack, A. N. Angelopoulos, A. Bajcsy, M. I. Jordan, J. Malik, "Conformal Decision Theory: Safe Autonomous Decisions from Imperfect Predictions," ICRA 2024. arXiv:2310.05921.

[26] A. N. Angelopoulos, S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," arXiv:2107.07511, 2021.